

AI ASSISTED CODING LAB – 1

Lab Experiment: Prompt Engineering – Improving Prompts and Context Management

Name : M Rithish Reddy

Roll No : 2503a52714

Batch No : 24

Question 1: Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Prompt : Generate a python function that checks whether a given number is a palindrome or not. If true then it is a palindrome and if no then it is not a palindrome

#code

```
1  def check_palindrome(number):
2      """Print whether the given number is a palindrome."""
3      text = str(number)
4      if text == text[::-1]:
5          print("It is a palindrome")
6      else:
7          print("It is not a palindrome")
8
9
10 number = int(input("Enter a number: "))
11 check_palindrome(number)
12
```

#output

```
rs/pc/OneDrive/AIAC/exp2.py
Enter a number: -7007
It is not a palindrome
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Use
rs/pc/OneDrive/AIAC/exp2.py
Enter a number: 7007
It is a palindrome
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Use
rs/pc/OneDrive/AIAC/exp2.py
Enter a number: 2007
It is not a palindrome
```

Logical Errors & Missing Edge Cases (Palindrome Program):

1. Negative numbers fail due to the - sign, not handled explicitly
2. No input validation (crashes on non-numeric input)
3. Function only prints, doesn't return True/False
4. Edge cases like 0 or trailing zeros not tested

Question 2: One-Shot Prompting (Factorial Calculation)

Write a **one-shot prompt** by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Prompt : Write a Python function called factorial that computes the factorial of a given number. Example: Input: 5 → Output: 120

Generate the function based on this example.

#code

```
def factorial(number):
    """Return the factorial of a non-negative integer."""
    if number < 0:
        raise ValueError("factorial is not defined for negative numbers")

    result = 1
    for value in range(2, number + 1):
        result *= value
    return result

user_input = int(input("Enter a number: "))
print(factorial(user_input))
```

#output

Comparison with zero-shot solution:

Zero-shot version usually just does the loop without checking negative numbers — it may crash unexpectedly or give wrong output for $n < 0$.

One-shot version (yours) explicitly raises a `ValueError` for negative numbers, making the error controlled and clear instead of a random crash.

Improvements in clarity & correctness:

```
Enter a number: 9
362880
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIAC/exp2.py
Enter a number: -4
Traceback (most recent call last):
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 13, in <module>
    print(factorial(user_input))
           ~~~~~~^~~~~~
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 4, in factorial
    raise ValueError("factorial is not defined for negative numbers")
ValueError: factorial is not defined for negative numbers
PS C:\Users\pc\OneDrive\AIAC> 4
4
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIAC/exp2.py
Enter a number: 11
39916800
```

1. Handles negative input properly with a meaningful error message
2. Iterative loop (not recursive) — safer for large numbers
3. Docstring added, explaining what the function does
4. Output matches the given example exactly ($5 \rightarrow 120$), confirming correctness

Question 3: Few-Shot Prompting (Armstrong Number Check)

Write a **few-shot prompt** by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number

- Input: 123 → Output: Not an Armstrong Number

prompt: Write a Python function called `check_armstrong` that checks whether a given number is an Armstrong number.

Examples: Input: 153 → Output: Armstrong Number,

Input: 370 → Output: Armstrong Number,

Input: 123 → Output: Not an Armstrong Number

Generate the function based on these examples.

#code

```
exp2.py > check_armstrong
1 def check_armstrong(number):
2     """Return whether number is an Armstrong number."""
3     digits = str(number)
4     power = len(digits)
5     return sum(int(digit) ** power for digit in digits) == number
6
7
8 number = int(input("Enter a number: "))
9 if check_armstrong(number):
10     print("Armstrong Number")
11 else:
12     print("Not an Armstrong Number")
```

#output

```

Enter a number: 141
Not an Armstrong Number
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter a number: -141
Traceback (most recent call last):
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 9, in <module>
    if check_armstrong(number):
        ~~~~~^~~~~~
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 5, in check_armstrong
    return sum(int(digit) ** power for digit in digits) == number
               ~~~~~^~~~~~
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 5, in <genexpr>
    return sum(int(digit) ** power for digit in digits) == number
               ~~~~~^~~~~~
ValueError: invalid literal for int() with base 10: '-'
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter a number: 371
Armstrong Number

```

Logical error found: The code doesn't handle negative numbers. `str(-141)` includes the `-` sign, and `int('-')` fails when iterating character-by-character — causing an unhandled crash instead of a clean "Not an Armstrong Number" response.

(Optional Extension)

Question 4: Context-Managed Prompting (Optimized Number Classification)

Design a **context-managed prompt** with clear instructions and constraints to generate an optimized Python program that classifies a number as **prime, composite, or neither**.

Prompt: Write a Python program that asks the user to enter a number and tells whether it is prime, composite, or neither. Make sure the code checks for valid input and uses an efficient method (don't check every number up to n , just check up to its square root). Also skip checking even numbers to make it faster.

#code

```

exp2.py > ...
1  import math
2
3
4  def classify_number(number):
5      """Return whether number is prime, composite, or neither."""
6      if number <= 1:
7          return "neither"
8      if number == 2:
9          return "prime"
0      if number % 2 == 0:
1          return "composite"
2
3      limit = math.isqrt(number)
4      divisor = 3
5      while divisor <= limit:
6          if number % divisor == 0:
7              return "composite"
8          divisor += 2
9      return "prime"
0
1
2  try:
3      number = int(input("Enter an integer: ").strip())
4  except ValueError:
5      print("Invalid input. Please enter a whole number.")
6  else:
7      print(f"{number} is {classify_number(number)}.")

```

#output

```

7exp2.py
Enter an integer: 8
8 is composite.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/Or
/exp2.py
Enter an integer: -5
-5 is neither.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/Or
/exp2.py
Enter an integer: 5
5 is prime.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/Or
/exp2.py
Enter an integer: 1
1 is neither.

```

Task analysis:

- 1.Optimized for efficiency — uses `math.isqrt(number)` for $O(\sqrt{n})$ time, checks only odd divisors after eliminating even numbers early.
- 2.Comparison with earlier strategies — zero-shot versions typically lacked validation and used slower $O(n)$ loops; this context-managed version is faster, safer, and handles edge cases (negative numbers, 0, 1) correctly out of the box.

Question 5: Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Prompt: Write a python function for perfect number that checks whether a given number is a perfect number or not if yes then it is a perfect number if no then it is not a perfect number

#code

```
exp2.py 7 is_perfect_number
1  def is_perfect_number(number):
2      """Return True if number equals the sum of its proper divisors."""
3      if number <= 1:
4          return False
5
6      divisor_sum = 1
7      for divisor in range(2, int(number ** 0.5) + 1):
8          if number % divisor == 0:
9              divisor_sum += divisor
10             paired_divisor = number // divisor
11             if paired_divisor != divisor:
12                 divisor_sum += paired_divisor
13
14     return divisor_sum == number
15
16
17 number = int(input("Enter a number: "))
18 if is_perfect_number(number):
19     print("It is a perfect number.")
20 else:
21     print("It is not a perfect number.")
```

#output

```

/exp2.py
Enter a number: 6
It is a perfect number.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive
/exp2.py
Enter a number: -6
It is not a perfect number.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive
/exp2.py
Enter a number: 6.0
Traceback (most recent call last):
  File "c:\Users\pc\OneDrive\AIAC\exp2.py", line 17, in <module>
    number = int(input("Enter a number: "))
ValueError: invalid literal for int() with base 10: '6.0'
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive
/exp2.py
Enter a number: 100
It is not a perfect number.

```

Missing Conditions / Inefficiencies:

Crashes on decimal input (6.0) — `int()` can't parse a string with a decimal point

No try/except for invalid/non-numeric input

Already uses $\sqrt{n}$ optimization (efficient), but no upper limit check for very large numbers

No explicit test/comment for edge case `number == 1`

Question 6: Few-Shot Prompting (Even or Odd Classification with Validation)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python program that determines whether a given number is even or odd, including proper input validation.

Examples:

1.Input: 8 → Output: Even

2.Input: 15 → Output: Odd

3.Input: 0 → Output: Even

#prompt: Write a Python function called `check_even_odd` that determines whether a given number is even or odd, including proper input validation. Examples: Input: 8 → Output: Even Input: 15 → Output: Odd Input: 0 → Output: Even Generate the function based on these examples.

#code

```

exp2.py > check_even_odd
1  def check_even_odd(number):
2      """Return whether an integer is even or odd."""
3      if isinstance(number, bool) or not isinstance(number, int):
4          raise ValueError("Please enter a valid integer.")
5
6      return "Even" if number % 2 == 0 else "Odd"
7
8
9  try:
10     user_number = int(input("Enter an integer: "))
11     print(check_even_odd(user_number))
12 except ValueError as error:
13     print(error)

```

#output

```

/exp2.py
Enter a number: 100
It is not a perfect number.
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: 89
Odd
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: 78
Even
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: -10
Even
PS C:\Users\pc\OneDrive\AIAC> & C:\Users\pc\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/pc/OneDrive/AIA
/exp2.py
Enter an integer: 23.8
invalid literal for int() with base 10: '23.8'

```

Short analysis:

Examples improved output clarity — AI matched exact string format ("Even"/"Odd") instead of returning booleans.

Including 0 as an example pushed the AI to handle the zero edge case correctly.

Negative numbers work correctly since Python's % handles negatives properly for even/odd checks.

Input validation was added proactively due to the task instruction, even without an explicit invalid-input example.

Conclusion:

This lab assignment provided hands-on experience with different prompt engineering strategies — zero-shot, one-shot, few-shot, and context-managed prompting — applied to common programming problems like palindrome checking, factorial calculation, Armstrong number verification, prime/composite classification, and perfect number checking. Through testing and comparing AI-generated code across these strategies, it became clear that prompt specificity directly impacts code quality: zero-shot prompts often produced functional but edge-case-fragile code, while few-shot and context-managed prompts — by providing examples and explicit constraints — consistently generated more robust, efficient, and well-validated programs. This exercise highlighted that clear, well-structured prompts are essential for extracting reliable and production-ready code from AI models.